DATA 260 Homework 8 - Rental Housing Listings

github.com/ishuapurva1996/data260-6102

Pragya Apurva | October 29, 2026

Fixed value	Configuration
SID4 / SEED	6102 / 6102
PORT_BASE / PREFIX	8702 / s6102
VERIFY_SEED / DOMAIN_ID	266102 / 6
Primary entity	Rental Housing Listings; CREATE_RENTAL and GET_RENTAL

Hardware: Apple MacBook Air, M4 (Mac16,12), 24 GiB memory. Runtime: Python 3.12.13. Local model: qwen2.5:7b, digest 845dbda0ea48ed749caa..., temperature 0, seed 6102, context 4096, output limit 256 tokens per stage; model timeout 180 seconds.

Tested source commit: `45d6327d222f8d17811a3e576f5bdf087ed2d5b9`. Kafka image: apache/kafka 3.9.2, pinned by digest. Full model/image details are in raw/system.json. RPC timeout is 8 seconds; retries allow three total processing attempts with 0.2-second backoff.

Submission tag: hw8. The tested source hash above identifies the code used for the recorded runs. The final tag points to the later evidence-only commit; its resolved target and complete post-tag smoke receipt are recorded in TAG_TARGET.json and post-tag/verification.json in the accompanying local upload package. No file claims to contain its own containing commit hash.

Repository continuity and access

This extends HW7 commit 1d5de17 in the same semester repository. New transport and stage modules live in shared root src/, code/ and scripts/. Pending HW7 report changes remain untouched in their original worktree. The repository link loaded successfully in GitHub on October 29. After the user completed identity verification, GitHub Manage access showed Sbnikitha and supriyaselvanganesan as collaborators under Direct access (2 collaborators). No invitation or access change was made. See raw/collaborator_access.json and the saved GitHub UI capture.

Capture types are explicit: Figure 1 and the HTTP/flow captures are screenshots of a real interactive PTY shell, displayed by a loopback terminal client. Source-code and earlier Part 3 figures are browser-rendered source/log views, not terminal captures. Each shows the student name. Raw JSON/CSV and timestamped logs provide the full records. Transport failure tests use fixture prose; Part 2 uses the real local model.

Part 1 - Rental request and response

The client sends a request to `s6102_entity_requests`. A separate worker validates it, changes its Python dictionary, and publishes a reply to `s6102_entity_responses`. The client waits for the matching reply. A correlation ID links those separate events.

Message contracts

```
Request:
{"correlation_id":"<caller ID>","reply_to":"s6102_entity_responses",
 "operation":"CREATE_RENTAL","data":{...rental fields...}}
CREATE response:
{"correlation_id":"<same ID>","operation":"CREATE_RENTAL",
 "success":true,"rental_id":"<worker UUID>","message":"Rental created"}
GET request data: {"rental_id":"<returned UUID>"}
GET response: {"correlation_id":"<same ID>","operation":"GET_RENTAL",
 "success":true,"rental":{"rental_id":"...",...validated fields...}}
Error: {"correlation_id":"<same ID>","operation":"...",
        "success":false,"error":"<clean explanation>"}
```

The six required fields are listingTitle, propertyAddress, submitterEmail, description, propertyType and termsAccepted. Existing RentalCreate rules enforce trimmed nonempty title/address, valid email, a description of at least 26 characters, an allowed property type and strict Boolean true for acceptance. Optional availableUnits keeps the existing validator. The worker generates a UUID; earlier SQL/Mongo IDs are unchanged.

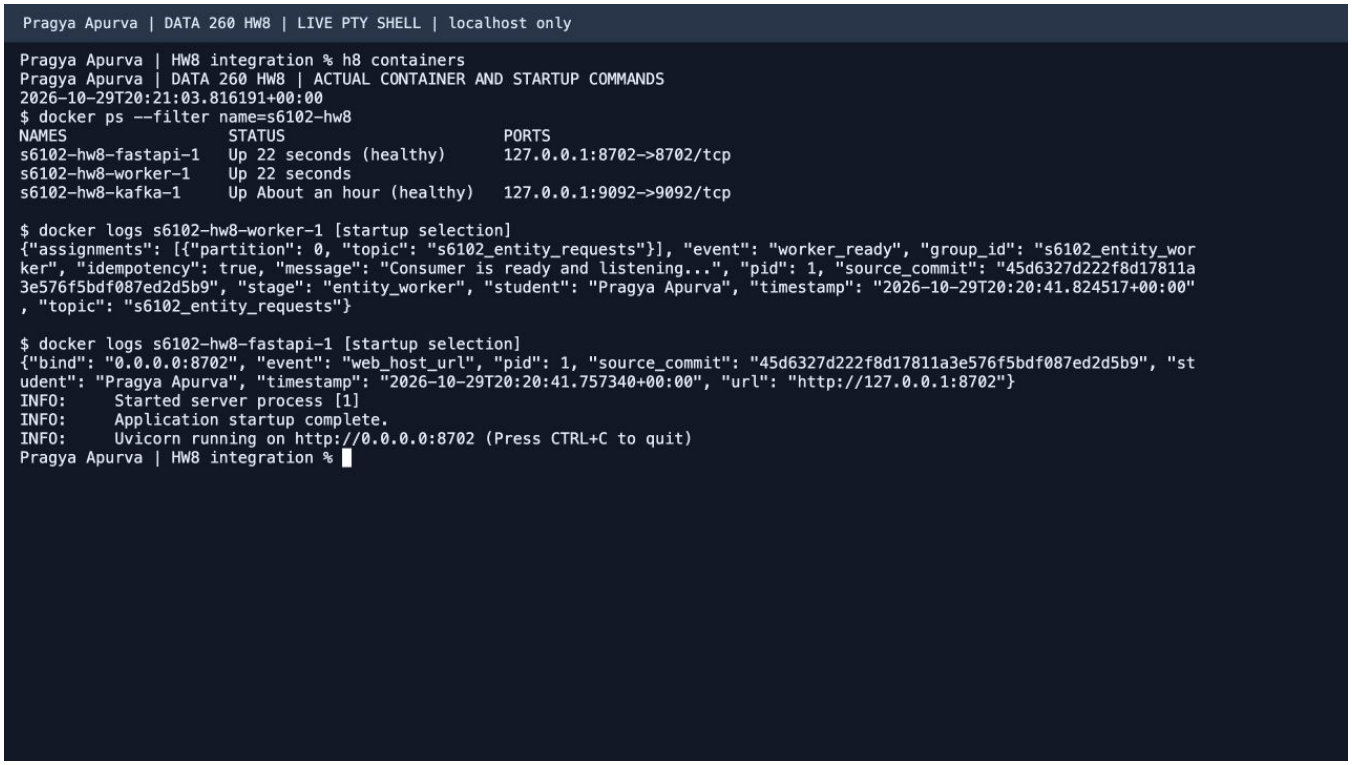


Figure 1. Actual PTY terminal capture: Kafka, worker and FastAPI containers, worker readiness, and real server/host URL startup logs.

Part 1 - Correlated client and demo

The client registers its pending future before publication and matches both correlation_id and operation. That prevents a delayed CREATE response from satisfying the later GET, which intentionally reuses the same journey ID.

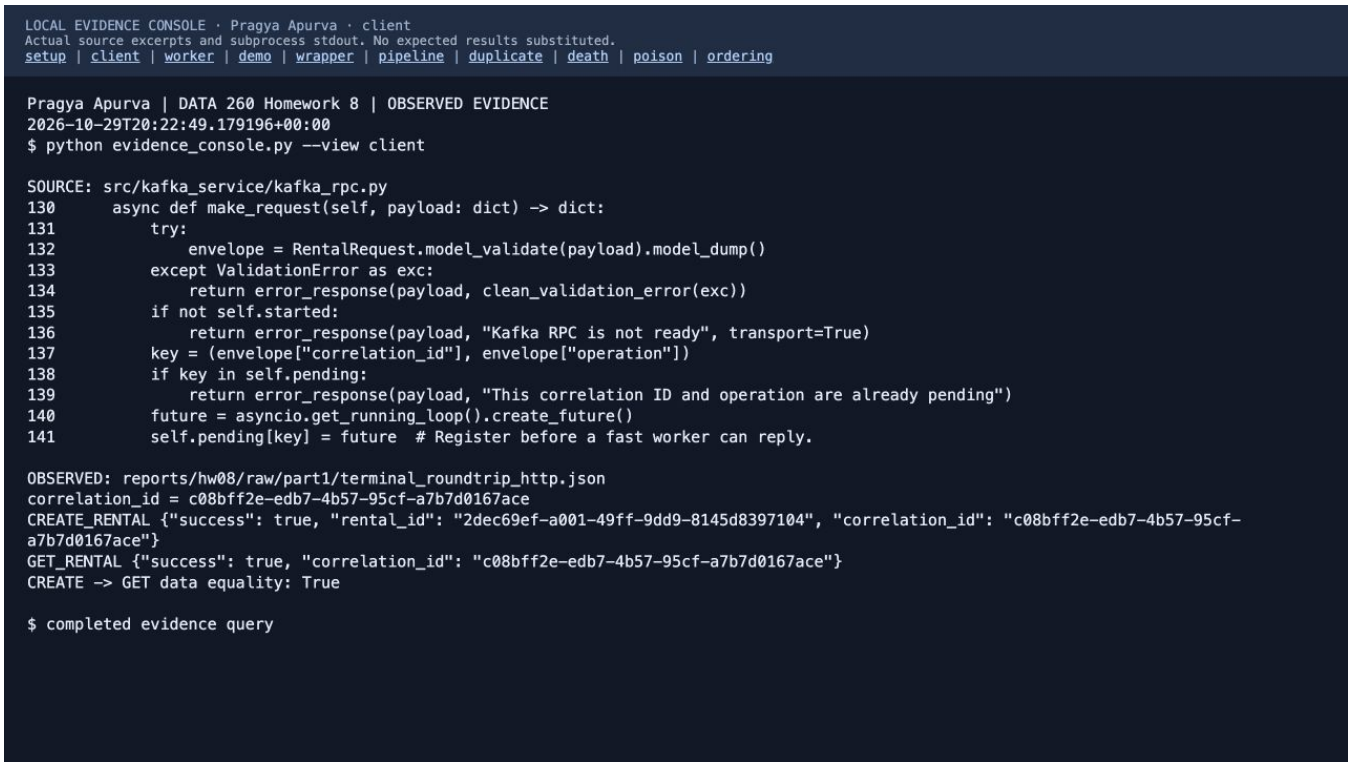


Figure 2. KafkaRPC source followed by the observed correlated responses. These responses came through real Kafka.

Part 1 - CREATE then GET demo

homework_demo.py creates one ID, extracts the worker UUID from CREATE, sends GET with that UUID and the same correlation ID, then checks every supplied rental field. The observed UUID was `2dec69ef-a001-49ff-9dd9-8145d8397104`. The observed journey ID was `c08bff2e-edb7-4b57-95cf-a7b7d0167ace`.

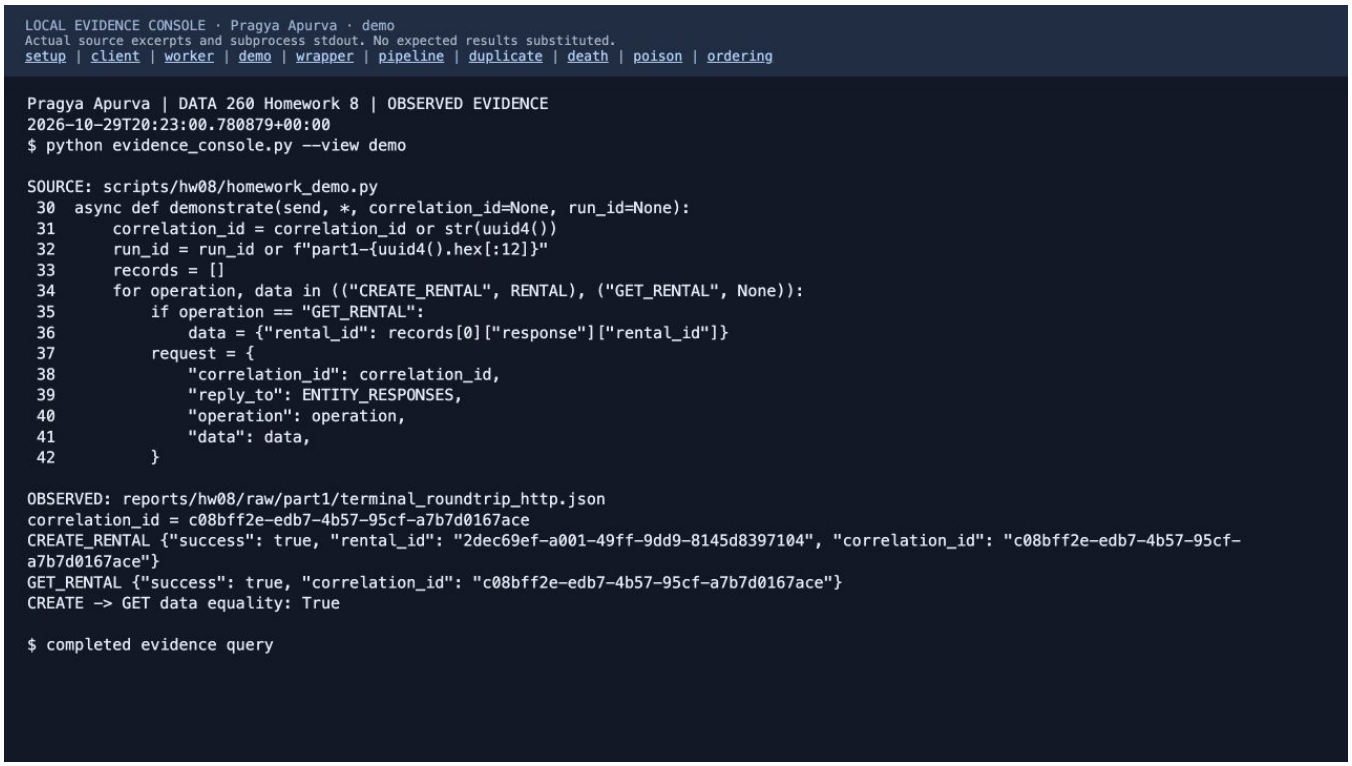


Figure 3. Browser-rendered demo source and observed HTTP results. Full pairs for this captured run are in raw/part1/terminal_roundtrip_http.json.

Part 1 - Live HTTP and account roles

This PTY command executed real HTTP logins and rental requests against port 8702. Passwords and bearer tokens were kept in memory and omitted from logs. CREATE returned the worker UUID; GET returned that rental with the same correlation ID and matching fields.

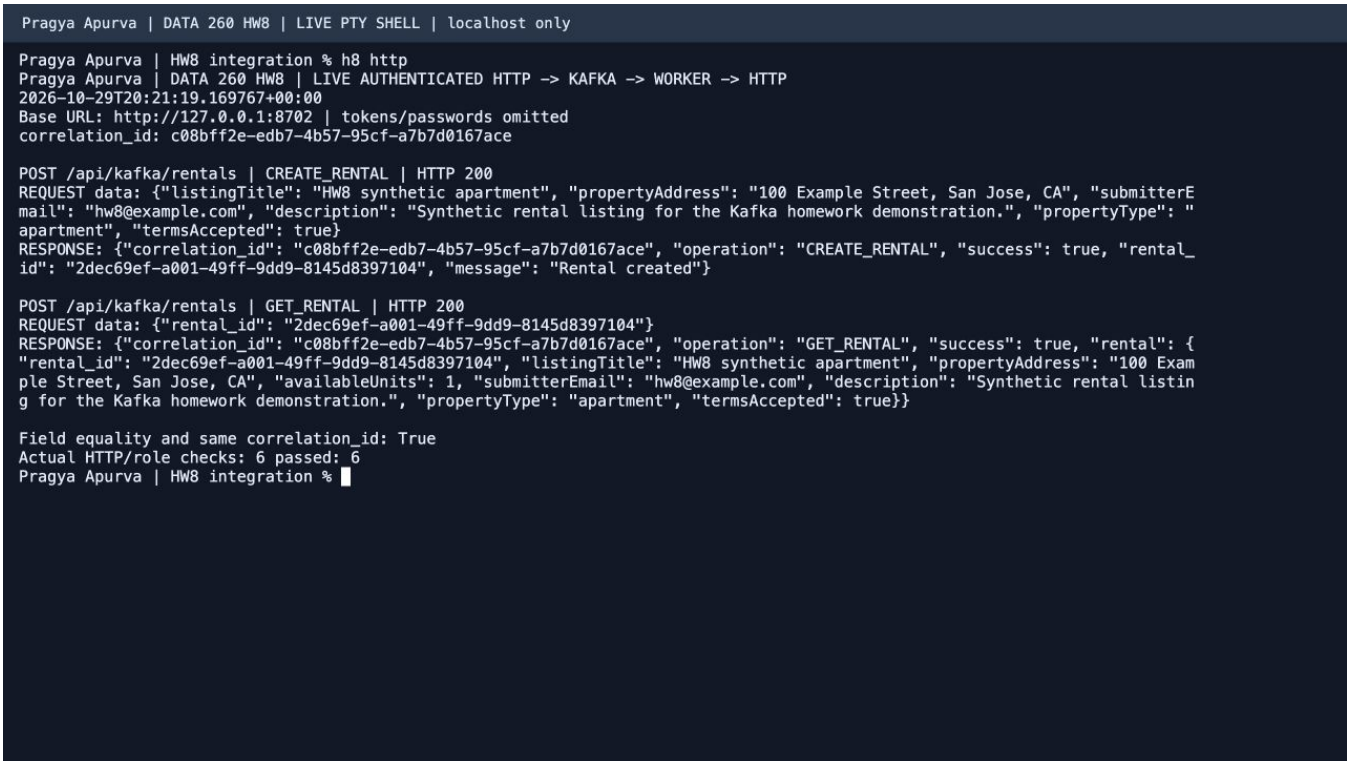


Figure 3a. Actual live PTY session executing the authenticated HTTP/Kafka CREATE and GET. This is not the browser log-viewer used for source excerpts.

Account or input	Observed HTTP result
Reader / writer / moderator login	200 / 200 / 200
Missing token (CREATE and GET), invalid token	401 / 401 / 401
Wrong password	401
Reader CREATE / GET	403 / 200
Writer CREATE / GET; moderator CREATE / GET	200 / 200; 200 / 200

The separate verifier repeats these checks with a fresh correlation ID. Its results are in raw/part1/http_auth.json and roundtrip_http.json; the captured terminal run uses the terminal_ prefixed files.

Part 1 - Worker and authenticated wrapper

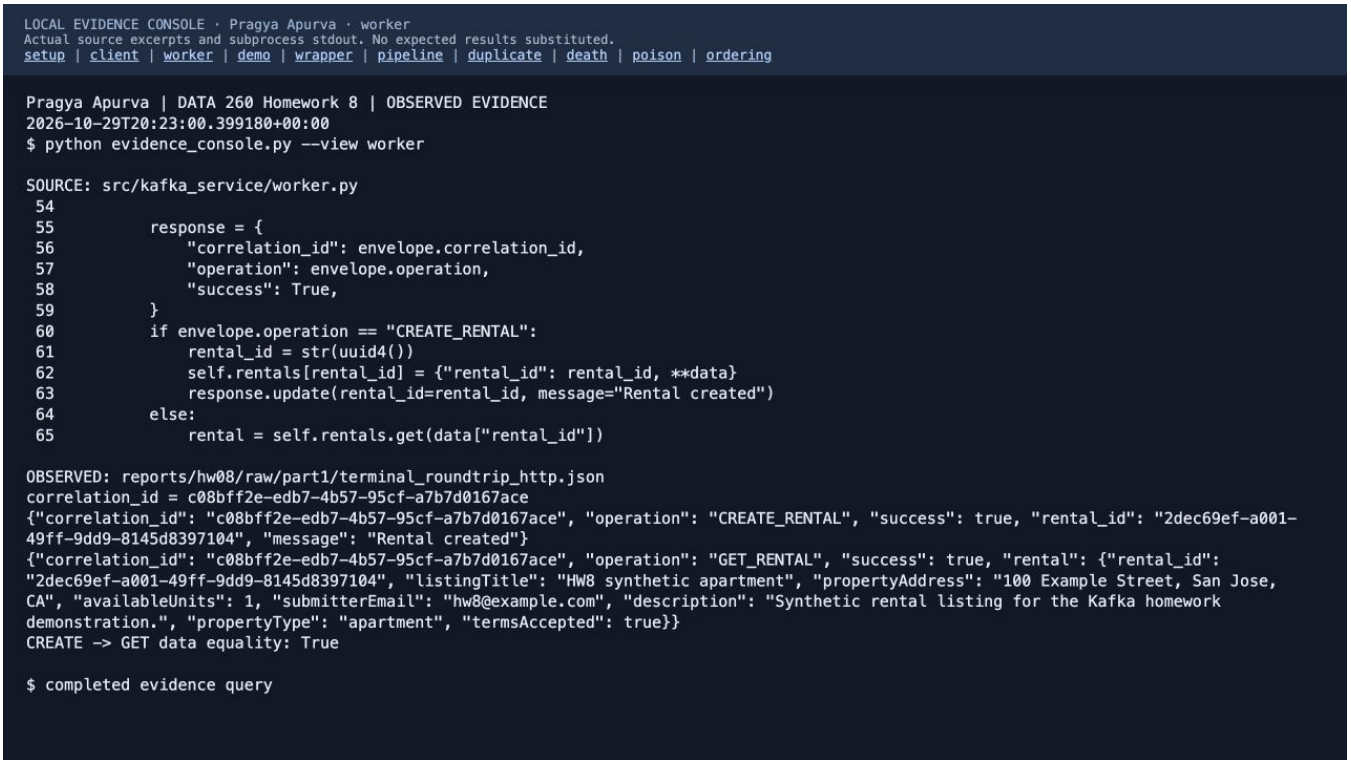


Figure 4. Worker UUID/dictionary code immediately followed by the generated UUID and retrieved rental values.

Part 1 - Authenticated API wrapper

The wrapper is added to the existing app factory, with startup/shutdown handlers for the RPC connection. It preserves the existing JWT user dependency. CREATE allows writer/moderator; GET allows an authenticated reader. Kafka errors return bounded transport errors; there is no direct-database fallback.

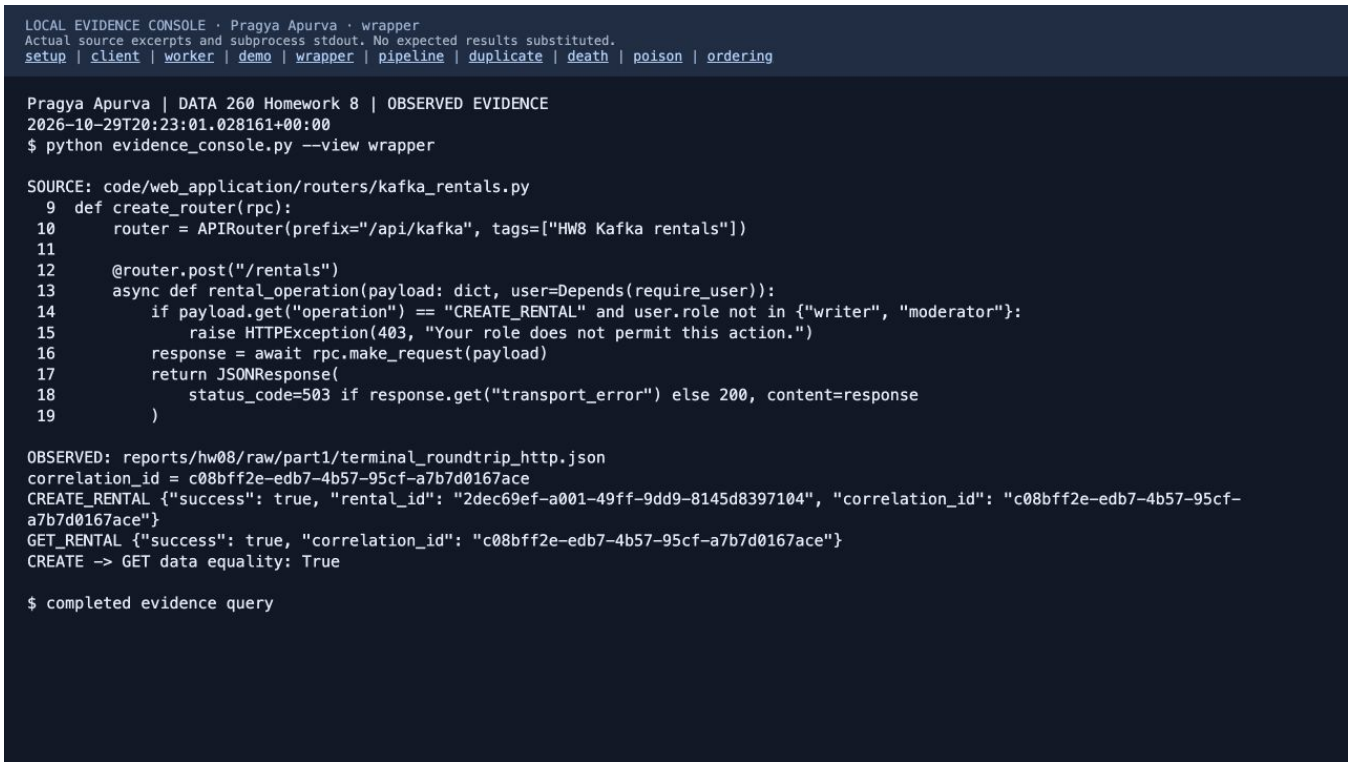


Figure 5. Browser-rendered wrapper source and observed authenticated HTTP results. Figure 3a and the flow capture provide actual terminal evidence.

Observed validation: missing required fields, termsAccepted=false, unknown operation and unknown UUID all returned success=false, a nonempty error and the unchanged caller ID. Offline tests also cover malformed inputs, stale replies, timeouts, send failure, cancellation, shutdown cleanup and wrapper authorization.

Part 1 - Complete flow and validation

The terminal query below joins real FastAPI/RPC and worker timestamps for the captured authenticated HTTP round trip. Each operation includes sending, worker receipt, processing, reply publication and client receipt/return. Kafka publication acknowledgment can reach the producer just after a consumer logs receipt, so event timestamps may slightly interleave across processes.

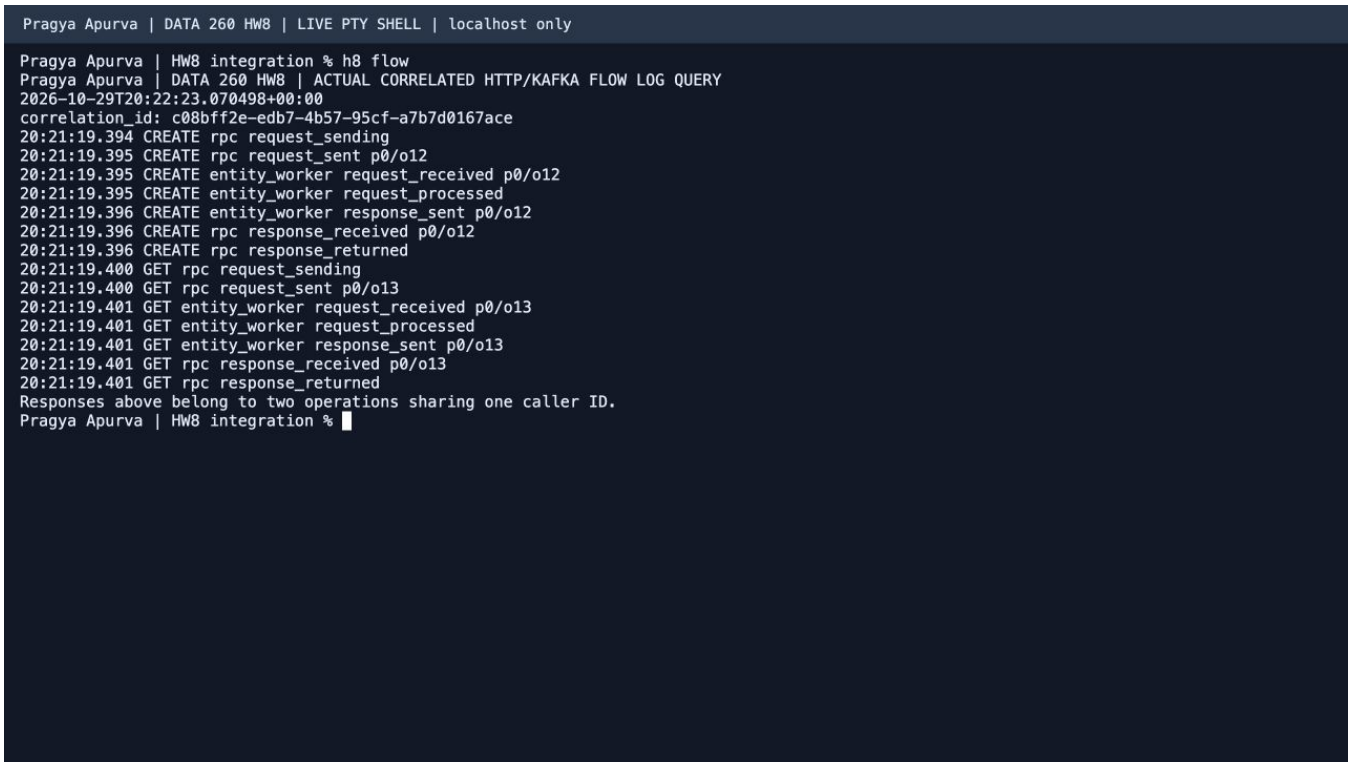


Figure 5a. Actual PTY query of the captured HTTP/Kafka flow: request publication, worker receive/process/reply, and API response receipt/return for both operations.

Live checks

Input	Observed outcome
CREATE_RENTAL	False; correlation preserved; listingTitle: Field required; propertyAddress: Field required; submitterEmail: Field required;
CREATE_RENTAL invalid terms	False; correlation preserved; termsAccepted: Value error, Please agree to the terms and conditions.
UNKNOWN	False; correlation preserved; Unknown operation
GET_RENTAL	False; correlation preserved; Rental not found

The dictionary and its response cache are intentionally worker-local. Restarting the entity worker loses rental entities; this exercise does not migrate them into SQL or MongoDB. Part 3 kills Writer, a different process whose recovery uses Kafka offsets and stage receipts.

Part 2 - Three separate processes

The route is `s6102_inbox -> Planner -> s6102_tasks -> Writer -> s6102_drafts -> Reviewer -> s6102_final`. inbox, drafts and final have one partition; tasks has two. Each stage owns a consumer group. Messages, rather than function calls, carry each handoff.

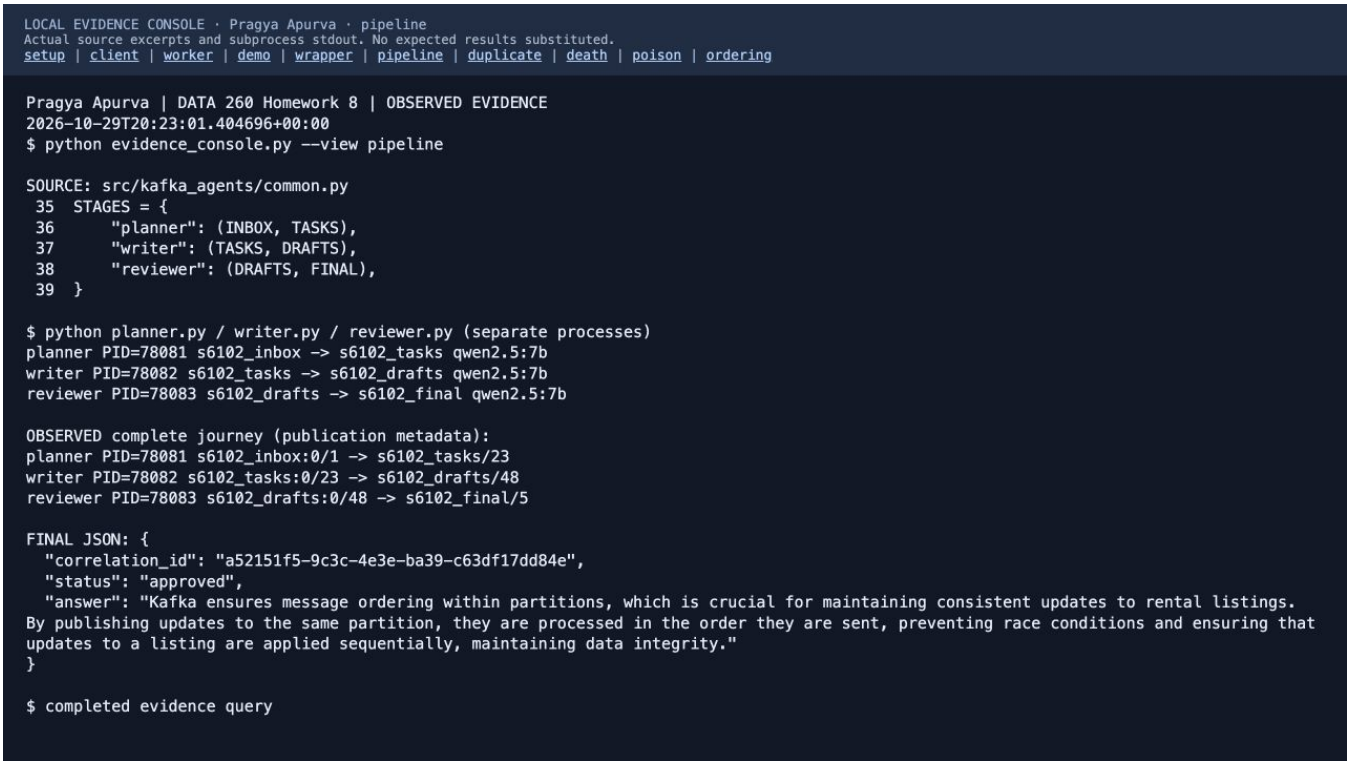


Figure 6. Stage routing code, three observed PIDs, publication journey and final status/answer from real qwen2.5:7b.

Stage output code

```
output = dict(data)  # preserve correlation_id and other identifiers
if self.stage == "planner":
    output["plan"] = content
elif self.stage == "writer":
    output["draft"] = content
    output["writer_instance"] = self.instance
else:
    output.update(status="approved", answer=content)
```

Observed final result

Status: approved. **Correlation ID:** `a52151f5-9c3c-4e3e-ba39-c63df17dd84e`.

Kafka ensures message ordering within partitions, which is crucial for maintaining consistent updates to rental listings. By publishing updates to the same partition, they are processed in the order they are sent, preventing race conditions and ensuring that updates to a listing are applied sequentially, maintaining data integrity.

Kafka retains a stage handoff until its consumer reads it. Planner can publish a plan while Writer is unavailable; Writer can restart without restarting Planner or Reviewer. The correlation ID remains a mechanical envelope field, so model-generated wording cannot change the journey identity. The observed answer is an example model output, not proof of distributed exactly-once processing.

Part 3 - Before/after failure results

Before-fix modes deliberately reproduce defects. Corrected runs use normal defaults. Every run uses a fresh group namespace primed to current topic ends; restart experiments keep the same group and stage state. All failure experiments use deterministic fixture prose to isolate Kafka behavior.

Failure scenario	Before fix: observed	After fix: observed
Duplicate delivery	Same CREATE replay: 2 dictionary rentals / 2 UUIDs. Pipeline replay: 2 final outputs.	1 dictionary rental; identical UUID returned twice. Pipeline replay: 1 final output.
Writer killed mid-work	Early commit; offset 6 was skipped on restart. 0 outputs in a 4-second observation window.	Offset 7 redelivered to a new PID; 1 output, then commit 8.
Poison message	Writer exited with code 1 on first invalid plan. Next same-partition task produced 0 outputs.	3 total failed attempts; 1 DLQ record; next task in the same partition produced 1 output.
2 partitions / 2 Writers	20/20 outputs across partitions 0 and 1, completed out of send order.	Rental key sent all 20 to partition 1; observed sequence was exactly 1-20.

Matching evidence: raw/part3/duplicate_{before,after}.jsonl; pipeline_duplicate_{before,after}.jsonl; consumer_death_{before,after}.jsonl; poison_{before,after}.jsonl; ordering_{unkeyed,keyed}.csv. Each row below pairs the relevant code with actual output.

Part 3 - Duplicate delivery

The entity cache key includes operation and correlation ID, with an input fingerprint. An identical CREATE returns the original UUID without adding a dictionary entry. GET with the same journey ID remains a separate operation. Reusing an operation/ID with different input returns a conflict. Pipeline receipts similarly preserve a stage result and suppress a published replay.

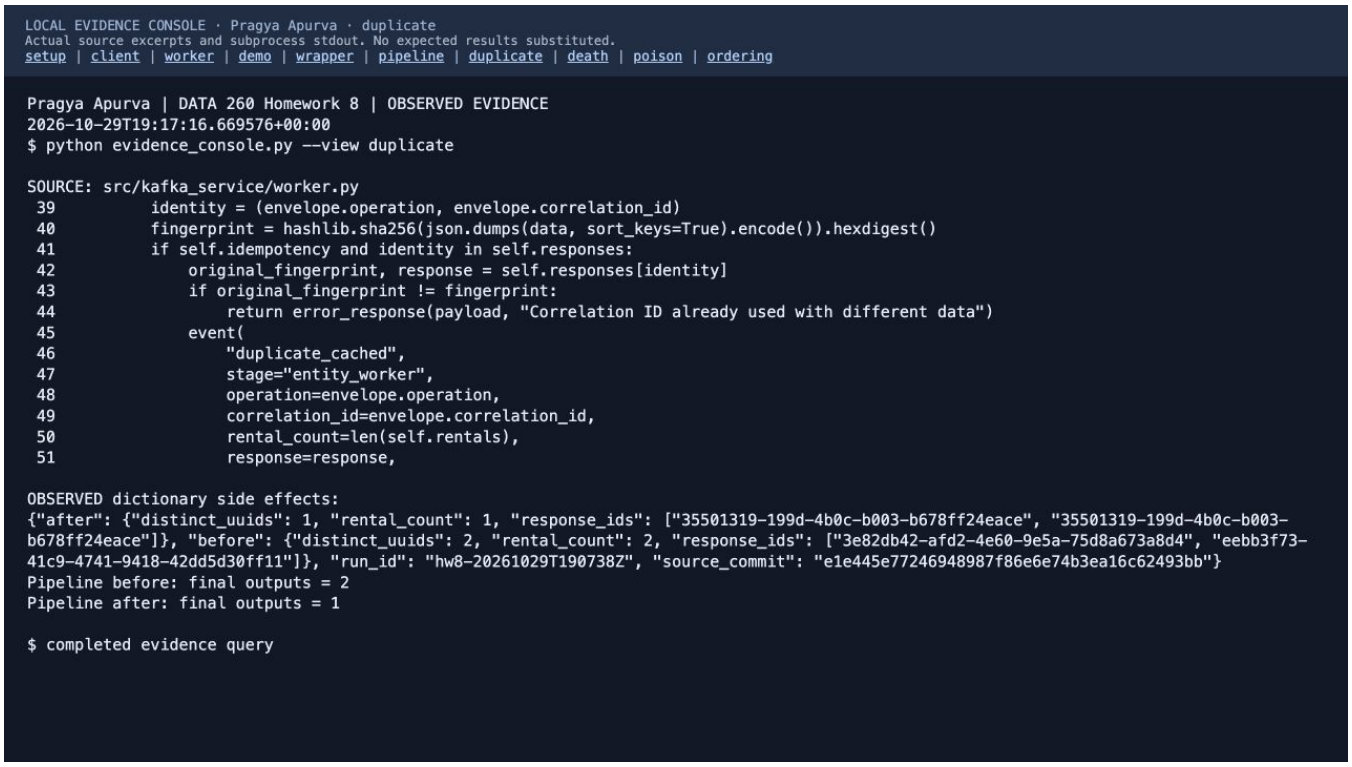


Figure 7. Idempotency code and measured dictionary/pipeline side effects. No extra logical result is inferred merely from seeing a duplicate log line.

Part 3 - Writer death and commit timing

The harness waits for processing_started, then sends SIGKILL to the Writer PID it launched. In the baseline, Writer committed before the delay. On restart the group resumed after that record. In normal mode the input remained uncommitted; the new process received the same topic, partition and offset, published its draft, and committed offset + 1.

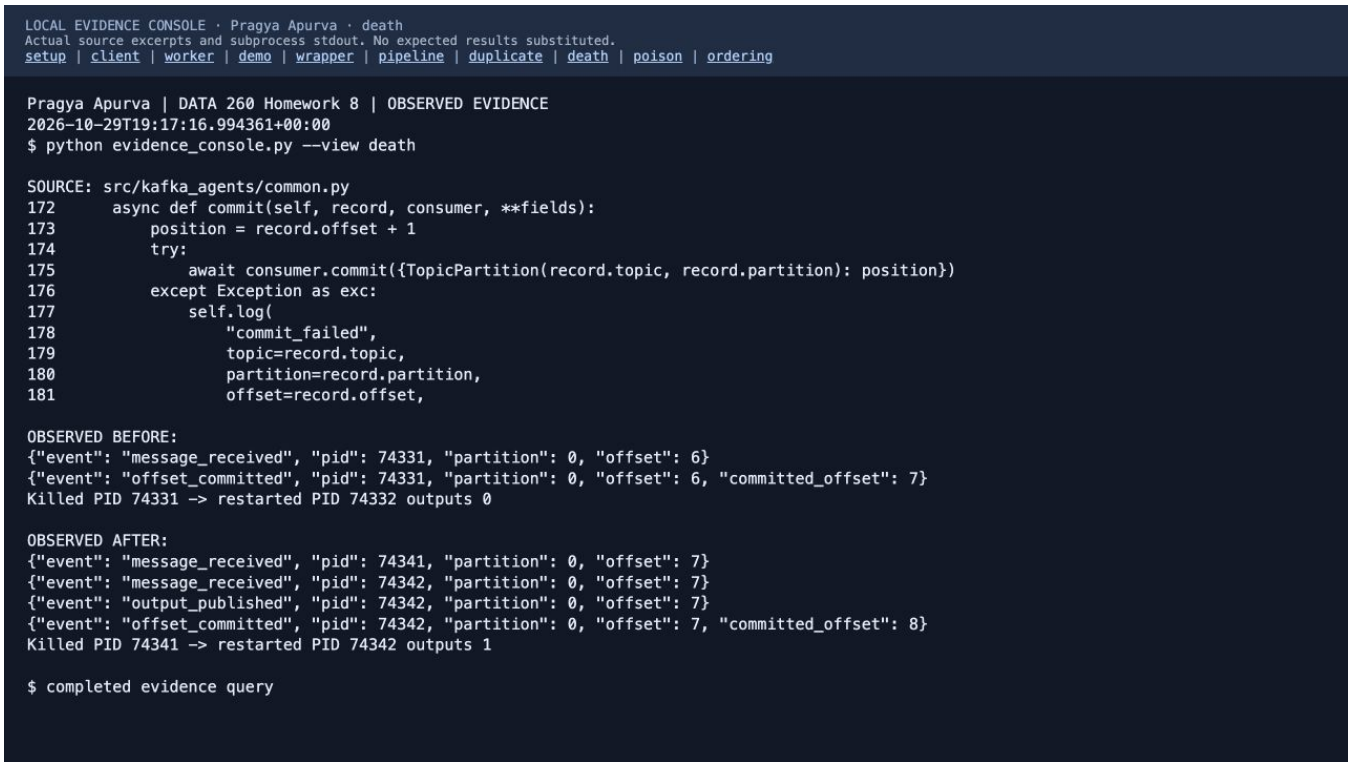


Figure 8. Commit code and before/after receive, publish and actual commit events. Different PIDs establish the restart.

Crash between publish and commit

A second controlled kill occurred after Kafka acknowledged publication but before Writer marked the receipt published. Restart recovered the cached output and could publish a repeated Kafka record. Reviewer suppressed the replay, leaving one final logical output. This demonstrates application deduplication over at-least-once delivery; SQLite and Kafka are not one atomic distributed transaction.

The first evidence harness stopped recovered Writer too quickly after seeing its output. The objective verifier rejected that evidence because no successful commit was logged. The harness now waits for offset_committed before stopping the process. The original run is preserved externally and its failed verifier output is included in RUN_LOG.txt.

Part 3 - Poison message and bounded retries

An invalid task with plan=null is followed by a valid task in the same source partition. Without handling, Writer exits and later work does not complete during observation. With the fix, the source record gets at most three total attempts, stored durably by group/topic/partition/offset. Exhaustion publishes the original payload, raw bytes, source metadata, error and attempt count to s6102_dead_letter. Only its acknowledgment allows the failed input offset to be committed.

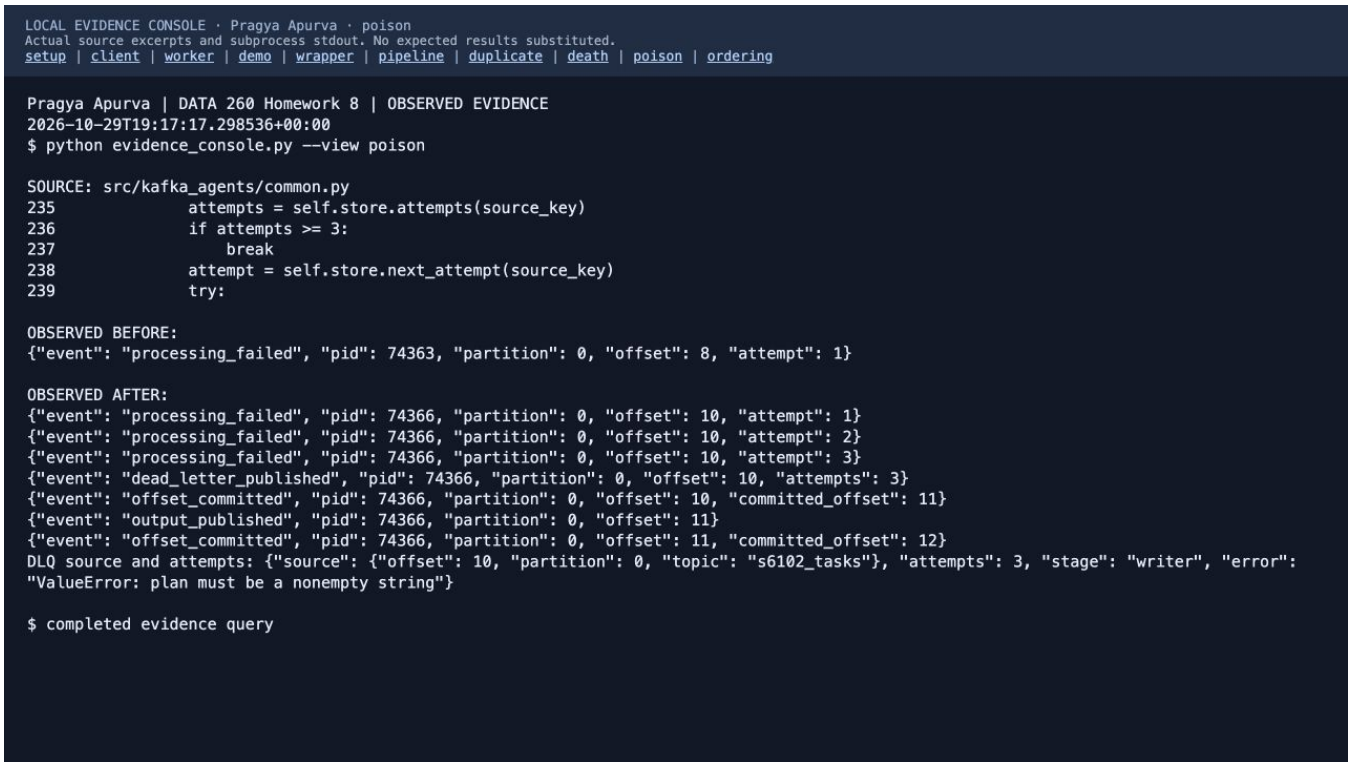


Figure 9. Three-attempt bound, actual failed attempts, DLQ publication and later same-partition completion. The observer also consumed the DLQ record.

JSON decoding happens inside the processing boundary, so malformed bytes can be sent to the DLQ instead of terminating the consumer. If output or DLQ publication fails, no successful input commit is recorded. Focused tests verify these boundaries and reopening receipt state after restart.

Part 3 - Partition keys and ordering

VERIFY_SEED=266102 generated 20 distinct correlation IDs for one synthetic rental and the saved delay schedule. The unkeyed baseline deliberately alternated explicit partitions; it is a controlled distribution, not a measurement of the producer default. Two Writer instances shared one group. The single-partition drafts stream supplied actual observation order.

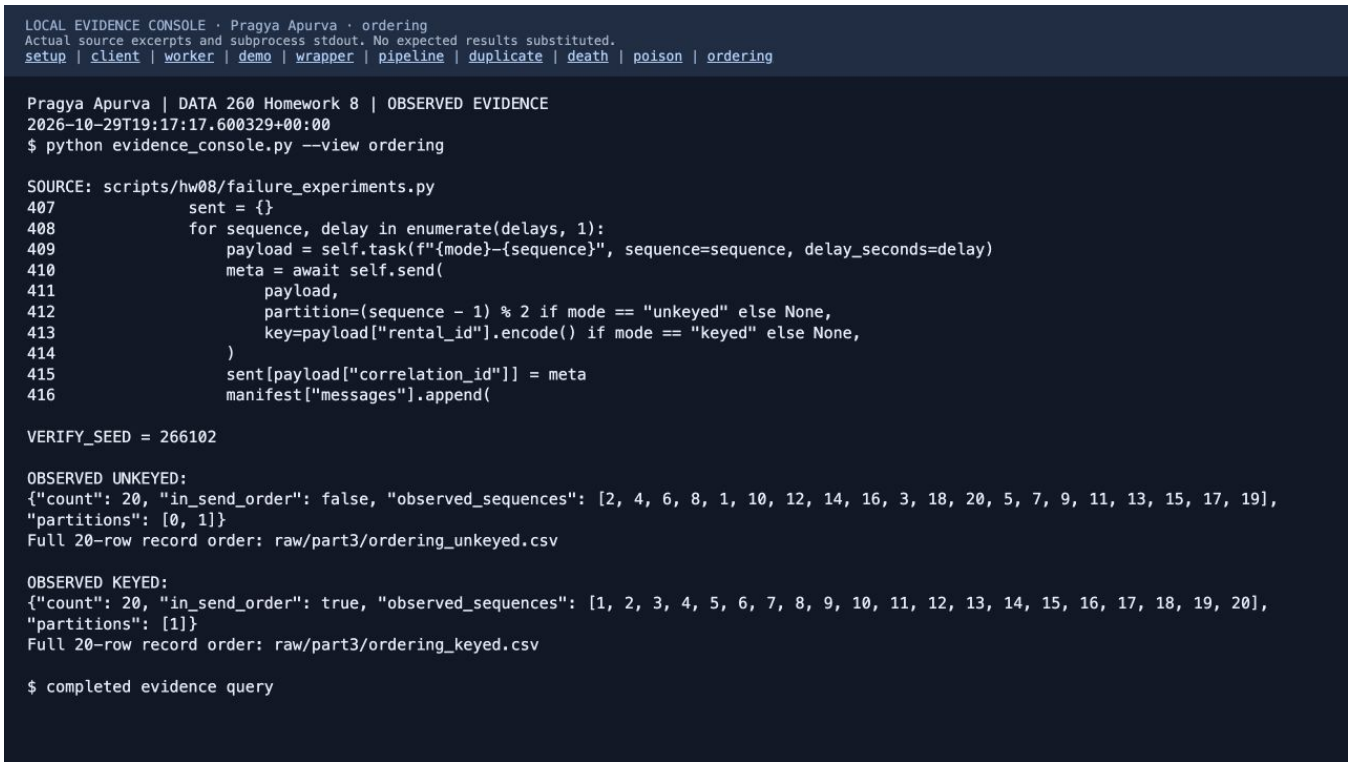


Figure 10. Keyed send code and observed 20-message summaries. Raw CSV files contain every record in observed publication order.

Unkeyed observed sequence: 2, 4, 6, 8, 1, 10, 12, 14, 16, 3, 18, 20, 5, 7, 9, 11, 13, 15, 17, 19.

Keyed observed sequence: 1-20, all in partition 1. The other Writer remains available but is idle for this one-rental workload. Kafka orders within a partition; a common rental key supplies that boundary. A single broker with replication factor 1 does not demonstrate broker high availability.

Comparison with the HW2 in-process graph

Two benefits: Writer can scale independently up to the available partitions, as the two-Writer run shows. A stage can restart independently while Kafka retains its handoff, as the Writer-death recovery shows. HW2 nodes instead waited on function calls inside one process.

Two new responsibilities: choose keys and process sequentially where per-rental order matters; manage duplicates, offset timing and poison records so retries do not lose work or repeat logical side effects. These responsibilities are explicit in the cache, receipts, manual commits and DLQ code.

Verification, limitations and AI use

The objective verifier passed 23 of 23 checks at the tested source commit. Named live checks verify topic counts, UUID CREATE/GET, identical replay, clean errors and a three-process real-model journey. Artifact checks evaluate the actual before/after data, all 20 ordering rows, redelivery offsets, commit acknowledgment and consumed DLQ records.

Focused verification with the existing MySQL service: all 68 tests passed, with zero skipped. All six database tables retained identical row counts and value fingerprints after rollback-isolated testing. Ruff and whitespace checks passed. A real broker-replacement check preserved the task end offset after Kafka log.dirs was mapped to its mounted volume; this checks local volume persistence, not replication or high availability.

Local acceptance passed for HTTP/Kafka, roles, database tests and the complete verifier. The hw8 tag identifies the final evidence commit. Its resolved target and repeated --require-ref hw8 result are kept externally to preserve the tagged checkout. Both required GitHub collaborators were confirmed with direct access after user identity verification. Nothing was pushed, published or submitted.

AI-use question 1 - Assistance and personal work

The user supplied the assignment, demos, configuration, plan and local setup authorization. The AI coordinator established contracts, integrated two subagents, ran experiments and acceptance checks, and drafted the report. The subagents implemented Parts 1 and 2 with focused tests. No independent student coding or manual validation is claimed for work not yet reviewed.

AI-use question 2 - Unsuitable output or independently checked result

The initial AI-written experiment harness treated an observed Writer output as enough evidence to stop the recovered process. That was unsuitable for proving commit timing. A separate check also found Kafka writing outside its configured Docker volume.

AI-use question 3 - Detection

The verifier examined actual offset_committed events and failed the initial recovery evidence. Inspecting the running Kafka configuration showed logs in /tmp/kafka-logs while the volume was mounted at /var/lib/kafka/data. These were checks of runtime behavior, not assumptions based on code appearance.

AI-use question 4 - Changes and why they work

The harness now waits for the commit acknowledgment. The repeated run records redelivery, output acknowledgment and offset + 1. Kafka logs now use the mounted volume; a real record survived container replacement. Initial failed evidence is retained separately. Authorized HTTP/role checks and all 68 tests now pass; Both required GitHub collaborators are now verified with direct access.

References: supplied DATA-260 Homework 8 PDF and both demo folders; existing HW2 graph/HW7 app. Technical interpretation was checked against aiokafka consumer and producer documentation: consumer.html and producer.html.